

Нахождение потомков элементов в JavaScript

Свойство firstElementChild

Свойство `firstElementChild` содержит первый дочерний элемент. Дочерними элементами считаются все теги, которые непосредственно расположены внутри блока. Если у элемента нет дочерних элементов - возвращается `null`.

Синтаксис

```
1 элемент.firstElementChild;
```

Пример

Получим содержимое первого потомка элемента:

```
1 <div id="parent">
2   <p>1</p>
3   <p>2</p>
4 </div>

1 let parent = document.querySelector('#parent');
2 let text = parent.firstElementChild.textContent;
3
4 console.log(text);
```

Результат выполнения кода:

```
1 '1'
```

Пример

А теперь у элемента нет дочерних элементов и поэтому выведется `null`:

```
1 <div id="parent"></div>

1 let parent = document.querySelector('#parent');
2 console.log(parent.firstElementChild);
```

Результат выполнения кода:

```
1 null
```

Свойство lastElementChild

Свойство `lastElementChild` хранит в себе последний дочерний элемент. Дочерними элементами считаются все теги, которые непосредственно расположены внутри блока. Если у элемента нет дочерних элементов - возвращается `null`.

Синтаксис

```
1 элемент.lastElementChild;
```

Пример

Получим содержимое последнего потомка элемента:

```
1 <div id="parent">
2   <p>1</p>
3   <p>2</p>
4 </div>

1 let parent = document.querySelector('#parent');
2 let text = parent.lastElementChild.textContent;
3
4 console.log(text);
```

Результат выполнения кода:

```
1 '2'
```

Пример

А теперь у элемента нет дочерних элементов и поэтому выведется null:

```
1 <div id="parent"></div>

1 let parent = document.querySelector('#parent');
2 console.log(parent.lastElementChild);
```

Результат выполнения кода:

```
1 null
```

Свойство children

Свойство children хранит в себе **псевдомассив** дочерних элементов. Дочерними элементами считаются все теги, которые непосредственно расположены внутри блока.

Синтаксис

```
1 элемент.children;
```

Пример

Давайте переберем в цикле всех потомков элемента и выведем их содержимое:

```
1 <div id="parent">
2   <p>1</p>
3   <p>2</p>
4   <p>3</p>
5   <p>4</p>
6   <p>5</p>
7 </div>
```

```
1 let parent = document.querySelector('#parent');
2 let elems = parent.children;
3
4 for (let elem of elems) {
5     console.log(elem.textContent);
6 }
```

Нахождение потомков элементов в JavaScript

Изучите теорию по следующим ссылкам:

[firstElementChild](#) [lastElementChild](#) [children](#)

№1

⊗jsPmSrED

Дан элемент #elem:

```
1 <ul id="elem">
2   <li>text</li>
3   <li>text</li>
4   <li>text</li>
5   <li>text</li>
6   <li>text</li>
7 </ul>
```

Найдите первого потомка этого элемента и сделайте его текст красного цвета.

№2

⊗jsPmSrED

Дан элемент #elem:

```
1 <ul id="elem">
2   <li>text</li>
3   <li>text</li>
4   <li>text</li>
5   <li>text</li>
6   <li>text</li>
7 </ul>
```

Найдите последнего потомка этого элемента и сделайте его текст красного цвета.

№3

⊗jsPmSrED

Дан элемент #elem:

```
1 <ul id="elem">
2   <li>text</li>
3   <li>text</li>
4   <li>text</li>
5   <li>text</li>
6   <li>text</li>
7 </ul>
```

Найдите всех потомков этого элемента и добавьте им в конец текст '!'.
Нахождение родителей элементов в JavaScript

Изучите теорию по следующим ссылкам:

[parentElement](#)

Свойство parentElement

Свойство parentElement содержит родительский элемент.

Синтаксис

```
1 элемент.parentElement;
```

Пример

Давайте получим родителя элемента #elem и выведем на экран его id:

```
1 <div id="parent">
2   <p id="elem"></p>
3 </div>
```

```
1 let elem = document.querySelector('#elem');
2 let id = elem.parentElement.id;
3
4 console.log(id);
```

Результат выполнения кода:

```
1 'parent'
```

Дан элемент #elem:

```
1 <div>
2   <ul>
3     <li>text</li>
4     <li>text</li>
5     <li id="elem">text</li>
6     <li>text</li>
7     <li>text</li>
8   </ul>
9 </div>
```

Найдите его родителя и задайте ему красную границу.

Поиск всех родителей элементов в JavaScript

Изучите теорию по следующим ссылкам:

[closest](#)

Метод closest

Метод `closest` ищет ближайший родительский элемент, подходящий под указанный CSS селектор, при этом сам элемент тоже включается в поиск.

Синтаксис

```
1 элемент.closest('селектор');
```

Пример

Давайте найдем среди родителей элемента элемент с классом `www`:

```
1 <div class="www" id="parent2">
2   <div class="ggg" id="parent1">
3     <p class="zzz" id="child"></p>
4   </div>
5 </div>
```

```
1 let elem = document.querySelector('#child');
2 let parent = elem.closest('.www');
3
4 console.log(parent.id);
```

Результат выполнения кода:

```
1 'parent2'
```

Пример

Давайте найдем среди родителей элемента элемент с классом `www`. В результате мы получим ссылку на сам элемент, так как он сам имеет этот класс:

```
1 <div class="www" id="parent2">
2   <div class="www" id="parent1">
3     <p class="www" id="child"></p>
4   </div>
5 </div>

1 let elem = document.querySelector('#child');
2 let parent = elem.closest('.www');
3 console.log(parent.id);
```

Результат выполнения кода:

```
1 | 'child'
```

№ 1

⊗ jsPmSrAP

Дан элемент:

```
1 <header>
2   <div>
3     <p>
4       <span id="elem"></span>
5     </p>
6   </div>
7 </header>
```

Найдите ближайшего родителя этого элемента, являющегося тегом `div`.

№ 2

⊗ jsPmSrAP

Дан элемент:

```
1 <header>
2   <div class="www">
3     <p class="www">
4       <span id="elem"></span>
5     </p>
6   </div>
7 </header>
```

Найдите ближайшего родителя этого элемента, являющегося элементом с классом `www`.

Поиск соседей элементов в JavaScript

Изучите теорию по следующим ссылкам:

[previousElementSibling](#) [nextElementSibling](#)

Свойство `previousElementSibling`

Свойство `previousElementSibling` содержит предыдущий элемент, находящийся в этом же родителе. Если такого элемента нет - возвращается `null`.

Синтаксис

```
1 элемент.previousElementSibling;
```

Пример

Дан элемент `#elem`. Давайте выведем текст его соседа сверху:

```
1 <p>sibling</p>
2 <p id="elem">elem</p>

1 let elem = document.querySelector('#elem');
2 let text = elem.previousElementSibling.textContent;
3
4 console.log(text);
```

Результат выполнения кода:

```
1 'sibling'
```

Пример

Если соседа сверху нет или он расположен не в родителе нашего элемента, также возвращается `null`:

```
1 <p>sibling</p>
2 <div>
3   <p id="elem">elem</p>
4 </div>

1 let elem = document.querySelector('#elem');
2 console.log(elem.previousElementSibling);
```

Результат выполнения кода:

```
1 null
```

Свойство nextElementSibling

Свойство `nextElementSibling` содержит следующий элемент, находящийся в этом же родителе. Если такого элемента нет - возвращается `null`.

Синтаксис

```
1 элемент.nextElementSibling;
```

Пример

Дан элемент `#elem`. Давайте выведем текст его соседа снизу:

```
1 <p id="elem">elem</p>
2 <p>sibling</p>

1 let elem = document.querySelector('#elem');
2 let text = elem.nextElementSibling.textContent;
3
4 console.log(text);
```

Результат выполнения кода:

```
1 'sibling'
```

Пример

Если соседа снизу нет или он расположен не в родителе нашего элемента, также возвращается `null`:

```
1 <div>
2   <p id="elem">elem</p>
3 </div>
4 <p>sibling</p>

1 let elem = document.querySelector('#elem');
2 console.log(elem.nextElementSibling);
```

Результат выполнения кода:

```
1 null
```

№ 1

✉jsfmsrso

Дан элемент `#elem`:

```
1 <ul>
2   <li>text</li>
3   <li>text</li>
4   <li id="elem">text</li>
5   <li>text</li>
6   <li>text</li>
7 </ul>
```


№2

©jsPmSrSb

Дан элемент #elem:

```
1 <ul>
2   <li>text</li>
3   <li>text</li>
4   <li id="elem">text</li>
5   <li>text</li>
6   <li>text</li>
7 </ul>
```

№3

©jsPmSrSb

Дан элемент #elem:

```
1 <ul>
2   <li>text</li>
3   <li>text</li>
4   <li id="elem">text</li>
5   <li>text</li>
6   <li>text</li>
7 </ul>
```

№4

©jsPmSrSb

Дан элемент #elem:

```
1 <ul>
2   <li>text1</li>
3   <li>text2</li>
4   <li id="elem">text3</li>
5   <li>text4</li>
6   <li>text5</li>
7 </ul>
```

Поиск элемента по id в JavaScript

Изучите теорию по следующим ссылкам:

[getElementById](#)

Метод getElementById

Метод `getElementById` позволяет получить элемент страницы по его атрибуту `id`. С полученным элементом можно будет производить различные манипуляции: менять его текст, атрибуты, CSS стили и так далее.

Синтаксис

```
1 document.getElementById(id элемента);
```

Пример

Давайте получим инпут по его id и поменяем ему содержимое атрибута value:

```
1 | <input id="elem" value="???">

1 | let elem = document.getElementById('elem');
2 | elem.value = '!!!';
```

Результат выполнения кода:

```
1 | <input id="elem" value="!!!">
```

№ 1

Дан элемент #elem:

```
1 | <div id="elem"></div>
```

Получите этот элемент с помощью метода getElementById и установите ему какой-нибудь текст.

Поиск элемента по имени тега в JavaScript

Изучите теорию по следующим ссылкам:

[getElementsByName](#)

Метод getElementsByTagName

Метод getElementsByTagName позволяет получить элементы страницы по имени тега. С полученными элементами можно производить различные манипуляции: менять их текст, атрибуты, CSS стили и так далее.

Синтаксис

```
1 | document.getElementsByTagName(имя тега);
```

Пример

Давайте получим все абзацы и изменим им текст с помощью свойства innerHTML:

```
1 | <p>elem 1</p>
2 | <p>elem 2</p>
3 | <p>elem 3</p>

1 | let elems = document.getElementsByTagName('p');
2 |
3 | for (let elem of elems) {
4 |   elem.innerHTML = '!!!';
5 | }
```

Результат выполнения кода:

```
1 <p>!!!</p>
2 <p>!!!</p>
3 <p>!!!</p>
```

№ 1

Дан список:

```
1 <ul>
2   <li>text</li>
3   <li>text</li>
4   <li>text</li>
5   <li>text</li>
6   <li>text</li>
7 </ul>
```

Поиск элемента по имени класса в JavaScript

Изучите теорию по следующим ссылкам:

[getElementsByClassName](#)

Метод `getElementsByClassName`

Метод `getElementsByClassName` позволяет получить элементы страницы по их классу, заданному в атрибуте `class`. С полученными элементами можно будет производить различные манипуляции: менять их текст, атрибуты, CSS стили и так далее.

Синтаксис

```
1 document.getElementsByClassName(имя класса);
```

Пример

Давайте получим все элементы с классом `www` и изменим им текст с помощью свойства `innerHTML`:

```
1 <p class="www">elem 1</p>
2 <p class="www">elem 2</p>
3 <p class="www">elem 3</p>

1 let elems = document.getElementsByClassName('www');
2
3 for (let elem of elems) {
4   elem.innerHTML = '!!!';
5 }
```

Результат выполнения кода:

```
1 <p class="www">!!!</p>
2 <p class="www">!!!</p>
3 <p class="www">!!!</p>
```

Даны элементы:

```
1 <p class="www">elem 1</p>
2 <p class="www">elem 2</p>
3 <p class="www">elem 3</p>
```

Поиск элементов внутри другого элемента в JavaScript

Пусть у нас есть некоторый элемент родитель, а внутри него другие элементы:

```
1 <div id="parent">
2   <div class="child">text</div>
3   <div class="child">text</div>
4   <div class="child">text</div>
5 </div>
```

Пусть ссылка на родитель уже получена в переменную:

```
1 let parent = document.querySelector('#parent');
```

В таком случае, при необходимости мы можем выполнить поиск по селектору внутри этого родителя, а не по всему документу. Для этого метод для поиска нужно применить не к `document`, а к переменной, содержащей нашего родителя.

Давайте для примера найдем внутри нашего родителя элементы с классом `child`:

```
1 let elems = parent.querySelectorAll('.child');
```



Дан элемент `#parent`:

```
1 <div id="parent">
2   <p class="www">text</p>
3   <p class="www">text</p>
4   <p class="www">text</p>
5   <p class="ggg">text</p>
6   <p class="ggg">text</p>
7   <p class="ggg">text</p>
8 </div>
```

```
1 let parent = document.querySelector('#parent');
```

Найдите внутри родителя элементы с классом `www` и запишите их в переменную `elems1`. Затем найдите внутри родителя элементы с классом `ggg` и запишите их в переменную `elems2`.